SIMATS ENGINEERING

ASSESSMENT TOOL 1 – Medium (Scenario Based Assessment)

COURSE CODE: CSA0910 **COURSE NAME: Programming in Java**

COURSE OUTCOME COVERED

CO2: *Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and generics to store, retrieve, and process groups of objects in web applications.(BL3,BL4)*

QUESTIONS: 5 **Total Marks: 100**

ANNEXURE A

Scenario Based Assessment

Code of Conduct:

I B.Priyanka (192421206) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
B. Priyanka

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis				
Application of Concepts				
Analytical & Decision-Making Skills				
Solution Design & Approach				
Clarity, Justification & Presentation				

Scenario Based Assessment

Q1: Student Registration System

Suppose that a university system stores student data, where each student must be unique and can enroll in multiple courses.

(a) Enumerate three Collection interfaces suitable for this system.

(b) Design a Collection structure (diagram/representation) to store students and their enrolled courses.

(c) Write a Java program using HashSet and HashMap with Generics to:

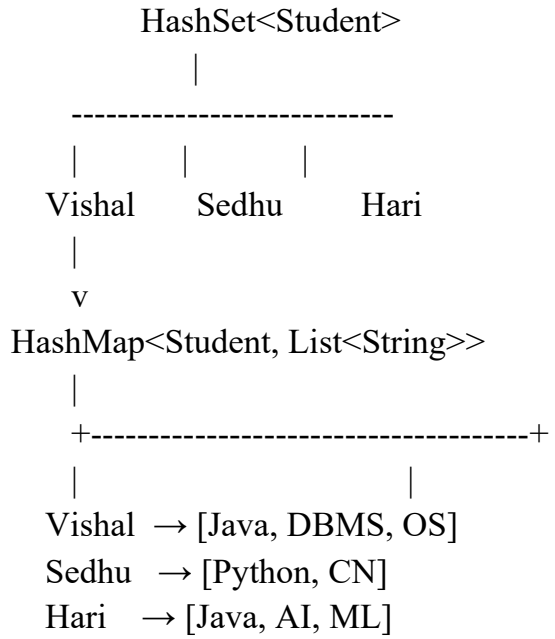
- Add students
- Map students to courses
- Display data using Iterator

ANSWER:

(a) Enumerate three Collection interfaces suitable for this system.

1. **Set** – Stores unique student records (prevents duplicate students).
2. **List** – Stores the list of courses enrolled by a student while maintaining insertion order.
3. **Map** – Maps each student to their enrolled courses using a key-value pair.

(b) Collection Structure (Diagram/Representation)



(c) Java Program using HashSet and HashMap with Generics

```
import java.util.*;

class Student {
   int id;
   String name;

   Student(int id, String name) {
      this.id = id;
```

```
        this.name = name;
    }

    // Override equals() and hashCode() for HashSet uniqueness
    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (!(obj instanceof Student))
            return false;

        Student s = (Student) obj;
        return id == s.id;
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }

    @Override
    public String toString() {
        return id + " - " + name;
    }
}

public class StudentRegistration {
    public static void main(String[] args) {
        HashSet<Student> students = new HashSet<>();

        Student s1 = new Student(101, "Vishal");
        Student s2 = new Student(102, "Sedhu");
        Student s3 = new Student(103, "Hari");
        students.add(s1);
        students.add(s2);
        students.add(s3);
        HashMap<Student, List<String>> studentCourses = new HashMap<>();
        studentCourses.put(s1, Arrays.asList("Java", "DBMS", "Operating System"));
        studentCourses.put(s2, Arrays.asList("Python", "Computer Networks"));
        studentCourses.put(s3, Arrays.asList("Java", "AI", "Machine Learning"));
        Iterator<Student> iterator = students.iterator();
        System.out.println("Student Registration Details:");
        System.out.println("----------------------------");
        while (iterator.hasNext()) {
            Student student = iterator.next();
            System.out.println("Student: " + student);
            System.out.println("Courses: " + studentCourses.get(student));
```

```
            System.out.println();
        }
    }
}
```

Sample Output:

```
Output
Student Registration Details:
----------------------------
Student: 101 - Vishal
Courses: [Java, DBMS, Operating System]

Student: 102 - Sedhu
Courses: [Python, Computer Networks]

Student: 103 - Hari
Courses: [Java, AI, Machine Learning]
```

Q2: E-Commerce Cart System

Suppose that an online shopping system maintains a cart where products are added, removed, and displayed in order.

(a) List three Collection classes applicable for cart management.

(b) Represent how cart items are stored using a List-based structure.

(c) Write a Java program using ArrayList and Iterator to:

- Add items
- Remove an item
- Display all items

ANSWER:

(a) List three Collection classes applicable for cart management.

1. ArrayList – Stores cart items in insertion order and allows fast access.
2. LinkedList – Efficient for frequent insertions and deletions.
3. Vector – Similar to ArrayList but synchronized (thread-safe).

(b) List-based Structure (Diagram/Representation)

Shopping Cart (ArrayList)

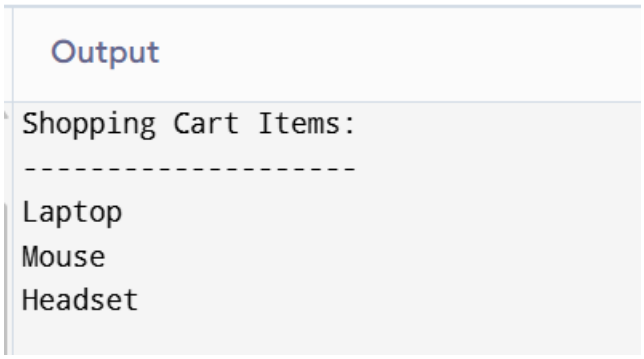
```
+---------+---------+---------+---------+
| Laptop  | Mouse   | Keyboard| Headset |
+---------+---------+---------+---------+
```

0 1 2 3

(c) Java Program using ArrayList and Iterator

```
import java.util.*;

public class Main {
   public static void main(String[] args) {

      // Create an ArrayList for cart items
      ArrayList<String> cart = new ArrayList<>();

      // Add items
      cart.add("Laptop");
      cart.add("Mouse");
      cart.add("Keyboard");
      cart.add("Headset");

      // Remove an item
      cart.remove("Keyboard");

      // Display items using Iterator
      Iterator<String> iterator = cart.iterator();

      System.out.println("Shopping Cart Items:");
      System.out.println("--------------------");

      while (iterator.hasNext()) {
         System.out.println(iterator.next());
      }
   }
}
```

SampleOutput:



Q3: Library Management System

Suppose that a library stores books with unique ISBN numbers and needs fast retrieval.
(a) Name three Map implementations in Java.
(b) Design a schema-like representation using Map for storing ISBN and book details.
(c) Write a Java program using HashMap to:

- Insert book details
- Retrieve a book
- Display all books

(a) Name three Map implementations in Java.

1. HashMap
2. TreeMap
3. LinkedHashMap

(b) Schema-like Representation
HashMap<ISBN, Book Name>

9781001 → Java Programming
9781002 → Database Management
9781003 → Computer Networks
9781004 → Operating Systems

(c) Java Program using HashMap (Without Comments)

```
import java.util.*;

public class Main {
   public static void main(String[] args) {

      HashMap<String, String> books = new HashMap<>();

      books.put("9781001", "Java Programming");
      books.put("9781002", "Database Management");
      books.put("9781003", "Computer Networks");
      books.put("9781004", "Operating Systems");

      String isbn = "9781002";

      System.out.println("Retrieved Book:");
      System.out.println(isbn + " : " + books.get(isbn));

      System.out.println("\nAll Books:");

      for (Map.Entry<String, String> entry : books.entrySet()) {
         System.out.println(entry.getKey() + " : " + entry.getValue());
```

```
    }
  }
}
```

Sample Output

```
Retrieved Book:
9781002 : Database Management

All Books:
9781002 : Database Management
9781001 : Java Programming
9781004 : Operating Systems
9781003 : Computer Networks

=== Code Execution Successful ===
```

Q4: Employee Data Processing

Suppose that a company processes employee records and filters employees based on salary.

(a) List three advantages of Generics in Collections.

(b) Represent how employee data is stored using List with Generics.

(c) Write a Java program using Iterator to:

- Store employee objects
- Display employees with salary > 50,000

(a) List three advantages of Generics in Collections.

1. Provides type safety.
2. Eliminates explicit type casting.
3. Improves code reusability and readability.

(b) Representation using List with Generics

List<Employee>

```
----------------------------------------
| ID | Name   | Salary                 |
----------------------------------------
|101 | Vishal | 45000                  |
|102 | Sedhu  | 60000                  |
|103 | Hari   | 75000                  |
----------------------------------------
```

(c) Java Program using Iterator

```
import java.util.*;
```

```
class Employee {
    int id;
    String name;
    double salary;

    Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }
}

public class Main {
    public static void main(String[] args) {

        List<Employee> employees = new ArrayList<>();

        employees.add(new Employee(101, "Vishal", 45000));
        employees.add(new Employee(102, "Sedhu", 60000));
        employees.add(new Employee(103, "Hari", 75000));

        Iterator<Employee> iterator = employees.iterator();

        System.out.println("Employees with Salary > 50000");

        while (iterator.hasNext()) {
            Employee emp = iterator.next();
            if (emp.salary > 50000) {
                System.out.println(emp.id + " " + emp.name + " " + emp.salary);
            }
        }
    }
}
```

Sample Output:

Output

```
Employees with Salary > 50000
102 Sedhu 60000.0
103 Hari 75000.0

=== Code Execution Successful ===
```

Q5: Online Voting System

Suppose that an online voting system ensures one vote per user and counts votes efficiently.
(a) Identify three suitable Collection types for this system.
(b) Design a structure using Set and Map for voters and vote counting.
(c) Write a Java program using HashSet and HashMap to:

- Record votes
- Prevent duplicate voting
- Display results

(a) Identify three suitable Collection types for this system.

1. HashSet
2. HashMap
3. TreeMap

(b) Structure using Set and Map

HashSet<Voters>

Vishal
Sedhu
Hari

HashMap<Candidate, Votes>

Alice → 2
Bob → 1

(c) Java Program using HashSet and HashMap (Without Comments)

```
import java.util.*;

public class Main {
   public static void main(String[] args) {

      HashSet<String> voters = new HashSet<>();
      HashMap<String, Integer> votes = new HashMap<>();

      String[][] votingData = {
         {"Vishal", "Alice"},
         {"Sedhu", "Bob"},
         {"Hari", "Alice"},
         {"Vishal", "Bob"}
      };
```

```
        for (String[] vote : votingData) {
            String voter = vote[0];
            String candidate = vote[1];

            if (voters.add(voter)) {
                votes.put(candidate, votes.getOrDefault(candidate, 0) + 1);
            } else {
                System.out.println(voter + " has already voted.");
            }
        }

        System.out.println("\nVoting Results:");

        for (Map.Entry<String, Integer> entry : votes.entrySet()) {
            System.out.println(entry.getKey() + " : " + entry.getValue());
        }
    }
}
```

Sample Output

Output

```
Vishal has already voted.

Voting Results:
Bob : 1
Alice : 2

=== Code Execution Successful ===
```